Prinzipien der Robotik

Embedded Systems Engineering – Vorlesung

Prof. Dr.-Ing. Sebastian Schlesinger

Fachbereich 2 – Duales Studium Wirtschaft & Technik
HWR Berlin

Wintersemester 2026

Gliederung

1 Sensorsignale verstehen

2 Signalverarbeitung

3 Zustandsschätzung

4 Kinematik & Bewegungsmodelle

5 Regelung (Control)

6 Lokalisierung & Mapping (SLAM)

7 Pfad- und Trajektorienplanung

8 Softwarearchitektur (ROS 2)

9 Echtzeit und Embedded-Aspekte

10 Unsicherheit & Verifikation

11 Zusammenfassung

Von der Modellierung zur Implementierung

Bisherige Vorlesung: Werkzeuge und Methoden der Modellierung

- Software Engineering, FSMs, State Charts
- Timed Automata, Verifikation mit UPPAAL
- Programmierung in C und Rust
- ROS 2 als Middleware

Von der Modellierung zur Implementierung

Bisherige Vorlesung: Werkzeuge und Methoden der Modellierung

- Software Engineering, FSMs, State Charts
- Timed Automata, Verifikation mit UPPAAL
- Programmierung in C und Rust
- ROS 2 als Middleware

Jetzt: Wie setzt man diese Werkzeuge ein, um einen realen Roboter zu bauen und zu steuern?

Von der Modellierung zur Implementierung

Bisherige Vorlesung: Werkzeuge und Methoden der Modellierung

- Software Engineering, FSMs, State Charts
- Timed Automata, Verifikation mit UPPAAL
- Programmierung in C und Rust
- ROS 2 als Middleware

Jetzt: Wie setzt man diese Werkzeuge ein, um einen realen Roboter zu bauen und zu steuern?

Kernfrage

Wie kommt ein mobiler Roboter von Sensordaten zu sicherer, zielgerichteter Bewegung – und wie verifizieren wir das?

Sensorsignale verstehen

Sensormodelle – Überblick

Ein Roboter nimmt die Welt ausschließlich über Sensoren wahr. Jeder Sensor liefert ein *Modell* der physikalischen Größe.

Inertialsensoren (IMU)

- Beschleunigungssensor: $a = F/m$
- Gyroskop: Winkelgeschwindigkeit ω
- 6-DOF (degrees of freedom): $[a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]^T$

a_i ist die Beschleunigung entlang der i-ten Achse (Bewegung im Raum), ω_i die Winkelgeschwindigkeit um die i-te Achse (Drehung im Raum).

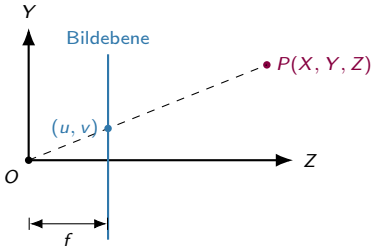
Entfernungssensoren

- Lidar / Ultraschall: Time-of-Flight
- $d = \frac{c \cdot \Delta t}{2}$
- Lidar: Laserpulse, 360-Scan
- Ultraschall: Schallkegel, geringere Auflösung

Sensormodell: Kamera – Projektionsmodell

Pinhole-Kameramodell: $(u, v, w) = K \cdot (X, Y, Z)$
(homogene Koordinaten) $\Rightarrow$ 2D-Bildkoordinaten
durch Normalisierung: $(u/w, v/w)$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{K \text{ (Kameramatrix)}} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$



- f_x, f_y: Brennweite in Pixel
- c_x, c_y: Bildhauptpunkt
- 3D-Punkt $(X, Y, Z) \mapsto$ 2D-Pixel (u, v)

Signalcharakteristika

Kein Sensor liefert perfekte Daten. Typische Störeinflüsse:

Rauschen

- Gaussian Noise: $n \sim \mathcal{N}(0, \sigma^2)$
- Signal-Rausch-Verhältnis (SNR)
- Thermisches Rauschen, Quantisierungsrauschen

Bias & Drift

- Konstanter Offset: $\hat{x} = x + b$
- Drift: $b(t)$ ändert sich langsam über Zeit
- Besonders kritisch bei Gyroskopen (IMU)

Quantisierung (ADC)

- Analog $\rightarrow$ Digital
- n Bit $\Rightarrow 2^n$ Stufen
- Auflösung: $\Delta = \frac{V_{\text{ref}}}{2^n}$
- 10-Bit ADC bei 3.3 V: $\Delta \approx 3.2\,\text{mV}$

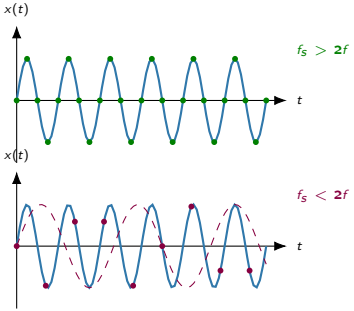
Abtasttheorem

Ein Signal mit maximaler Frequenz $f_{\max}$ kann exakt rekonstruiert werden, wenn die Abtastrate $f_s > 2 \cdot f_{\max}$ beträgt.

Aliasing: Wird zu langsam abgetastet ($f_s \leq 2f_{\max}$), erscheinen hochfrequente Komponenten als niederfrequente Geistersignale.

Praxis:

- IMU: typisch 100 Hz – 1000 Hz
- Lidar: 5 Hz – 40 Hz (Scanrate)
- Kamera: 30 Hz – 60 Hz (Framerate)



Rauschen – Warum ist es normalverteilt?

Zentraler Grenzwertsatz (Central Limit Theorem):
Seien $X_1, X_2, \ldots, X_n$ unabhängige Zufallsvariablen mit $\mathrm{E}[X_i] = \mu_i$ und $\mathrm{Var}[X_i] = \sigma_i^2$. Dann konvergiert die Summe

$$S_n = \sum_{i=1}^{n} X_i$$

für $n \to \infty$ gegen eine Normalverteilung – unabhängig von der Verteilung der einzelnen X_i.

Anwendung auf Sensoren:

- Ein Messwert wird von vielen kleinen, unabhängigen Störquellen beeinflusst (Temperatur, Elektronik, Quantisierung, Vibrationen, ...)
- Jede Störquelle X_i hat eigene Verteilung
- Deren Summe $\Rightarrow$ Gauß-Verteilung: $n \sim \mathcal{N}(0, \sigma^2)$

Varianzreduktion durch Mittelung

Gesetz der großen Zahlen – quantitativ:
Seien $X_1, \ldots, X_N$ i.i.d. mit $\mathrm{E}[X_i] = \mu$ und $\mathrm{Var}[X_i] = \sigma^2$. Der Mittelwert ist

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^{N} X_i$$

Varianzreduktion durch Mittelung

Gesetz der großen Zahlen – quantitativ:
Seien $X_1, \ldots, X_N$ i.i.d. mit $\mathrm{E}[X_i] = \mu$ und $\mathrm{Var}[X_i] = \sigma^2$. Der Mittelwert ist

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^{N} X_i$$

Erwartungswert: $\mathrm{E}[\bar{X}_N] = \frac{1}{N} \sum_{i=1}^{N} \mathrm{E}[X_i] = \mu$ (unverzerrter Schätzer)

Varianzreduktion durch Mittelung

Gesetz der großen Zahlen – quantitativ:
Seien $X_1, \ldots, X_N$ i.i.d. mit $\mathrm{E}[X_i] = \mu$ und $\mathrm{Var}[X_i] = \sigma^2$. Der Mittelwert ist

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^{N} X_i$$

Erwartungswert: $\mathrm{E}[\bar{X}_N] = \frac{1}{N} \sum_{i=1}^{N} \mathrm{E}[X_i] = \mu$ (unverzerrter Schätzer)
Varianz:

$$\mathrm{Var}[\bar{X}_N] = \mathrm{Var}\left[\frac{1}{N} \sum_{i=1}^{N} X_i\right] = \frac{1}{N^2} \sum_{i=1}^{N} \mathrm{Var}[X_i] = \frac{N\,\sigma^2}{N^2} = \frac{\sigma^2}{N}$$

Varianzreduktion durch Mittelung

Gesetz der großen Zahlen – quantitativ:
Seien $X_1, \ldots, X_N$ i.i.d. mit $\mathrm{E}[X_i] = \mu$ und $\mathrm{Var}[X_i] = \sigma^2$. Der Mittelwert ist

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^{N} X_i$$

Erwartungswert: $\mathrm{E}[\bar{X}_N] = \frac{1}{N} \sum_{i=1}^{N} \mathrm{E}[X_i] = \mu$ (unverzerrter Schätzer)
Varianz:

$$\mathrm{Var}[\bar{X}_N] = \mathrm{Var}\left[\frac{1}{N} \sum_{i=1}^{N} X_i\right] = \frac{1}{N^2} \sum_{i=1}^{N} \mathrm{Var}[X_i] = \frac{N\,\sigma^2}{N^2} = \frac{\sigma^2}{N}$$

Kernaussage

Mittelung über N Messwerte reduziert die Varianz um den Faktor N.
$\Rightarrow$ Gleitender Mittelwert = direkter Einsatz dieses Prinzips!

Vom Gesetz der großen Zahlen zum Moving Average

Moving Average als Fenster über die letzten N Werte:

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k]$$

Falls $x[n] = s[n] + \epsilon[n]$ mit Signal s und Rauschen $\epsilon[n] \sim \mathcal{N}(0, \sigma^2)$:

Signal bleibt erhalten (falls s langsam):

$$\mathrm{E}[y[n]] \approx s[n]$$

(Annahme: s ändert sich wenig im Fenster)

Rauschen wird reduziert:

$$\mathrm{Var}[y[n]] = \frac{\sigma^2}{N}$$

Standardabweichung: $\sigma_y = \frac{\sigma}{\sqrt{N}}$

Trade-off

Größeres $N \Rightarrow$ bessere Glättung, aber: Verzögerung $\frac{N-1}{2}$ Samples und Verlust schneller Signaländerungen. $\Rightarrow$ N muss zum System passen!

Signalverarbeitung

Filter – Grundlagen

Rohe Sensordaten sind verrauscht $\Rightarrow$ Filterung nötig.

Moving Average (Gleitender Mittelwert):

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k]$$

- Einfach zu implementieren
- Glättet Rauschen
- Verzögerung: $\frac{N-1}{2}$ Samples

Low-Pass / High-Pass (1. Ordnung):

$$y[n] = \alpha \cdot x[n] + (1-\alpha) \cdot y[n-1]$$

- $\alpha \in (0, 1]$: Glättungsfaktor
- α klein $\Rightarrow$ starke Glättung
- High-Pass: $y_{\mathrm{HP}} = x - y_{\mathrm{LP}}$

Implementierung (C)

```c
float lp_filter(float x, float y_prev, float alpha) {
    return alpha * x + (1.0f - alpha) * y_prev;
}
```

Low-Pass-Filter – Herleitung aus der Physik

Analogie: RC-Tiefpassfilter (Elektronik).
Die Spannung über dem Kondensator folgt der DGL:

$$RC\,\frac{dy}{dt} + y(t) = x(t)$$

Diskretisierung (Euler-Vorwärts, Schrittweite Δt):

$$RC\,\frac{y[n] - y[n-1]}{\Delta t} + y[n] = x[n]$$

Low-Pass-Filter – Herleitung aus der Physik

Analogie: RC-Tiefpassfilter (Elektronik).
Die Spannung über dem Kondensator folgt der DGL:

$$RC\,\frac{dy}{dt} + y(t) = x(t)$$

Diskretisierung (Euler-Vorwärts, Schrittweite Δt):

$$RC\,\frac{y[n] - y[n-1]}{\Delta t} + y[n] = x[n]$$

Umstellen nach $y[n]$:

$$y[n]\left(1 + \frac{RC}{\Delta t}\right) = x[n] + \frac{RC}{\Delta t}\,y[n-1]$$

$$y[n] = \underbrace{\frac{\Delta t}{RC + \Delta t}}_{\alpha}\,x[n] + \underbrace{\frac{RC}{RC + \Delta t}}_{1-\alpha}\,y[n-1]$$

Low-Pass-Filter – Herleitung aus der Physik

Analogie: RC-Tiefpassfilter (Elektronik).
Die Spannung über dem Kondensator folgt der DGL:

$$RC\,\frac{dy}{dt} + y(t) = x(t)$$

Diskretisierung (Euler-Vorwärts, Schrittweite Δt):

$$RC\,\frac{y[n] - y[n-1]}{\Delta t} + y[n] = x[n]$$

Umstellen nach $y[n]$:

$$y[n]\left(1 + \frac{RC}{\Delta t}\right) = x[n] + \frac{RC}{\Delta t}\,y[n-1]$$

$$y[n] = \underbrace{\frac{\Delta t}{RC + \Delta t}}_{\alpha}\,x[n] + \underbrace{\frac{RC}{RC + \Delta t}}_{1-\alpha}\,y[n-1]$$

Ergebnis: Exponential Moving Average (EMA)

$y[n] = \alpha \cdot x[n] + (1-\alpha) \cdot y[n-1]$ mit $\alpha = \frac{\Delta t}{RC+\Delta t} \in (0,1)$

Low-Pass: Frequenzverhalten und α

Zeitkonstante: $\tau = RC$ bestimmt die Grenzfrequenz $f_c = \frac{1}{2\pi\tau}$.

α groß (z. B. 0.8):

- τ klein $\Rightarrow f_c$ hoch
- Wenig Glättung, schnelle Reaktion
- Mehr Rauschen durchgelassen

α klein (z. B. 0.05):

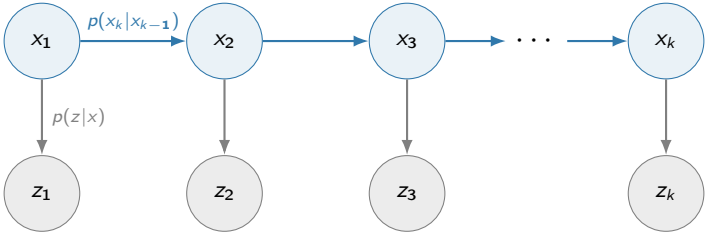
- τ groß $\Rightarrow f_c$ niedrig
- Starke Glättung, langsame Reaktion
- Rauschen stark gedämpft

High-Pass: $y_{\mathrm{HP}}[n] = x[n] - y_{\mathrm{LP}}[n]$ – lässt schnelle Änderungen durch, blockiert Drift ($\Rightarrow$ Gyroskop!).

Complementary Filter = LP auf Acc + HP auf Gyro

Genau das nutzt der Complementary Filter (kommt gleich).

Idee: Es gibt einen verborgenen Zustand x_k, den wir nicht direkt sehen – nur über verrauschte Beobachtungen z_k.



Zwei Grundannahmen (Markov-Eigenschaft):

1. x_k hängt *nur* von x_{k-1} ab: $p(x_k|x_1, \ldots, x_{k-1}) = p(x_k|x_{k-1})$
2. z_k hängt *nur* von x_k ab: $p(z_k|x_1, \ldots, x_k) = p(z_k|x_k)$

Vom HMM zum Kalman-Filter

Ein HMM wird durch drei Größen definiert:

Diskretes HMM (endliche Zustände):

- Transition: $A_{ij} = p(x_k{=}j \mid x_{k-1}{=}i)$
- Emission: $B_{jm} = p(z_k{=}m \mid x_k{=}j)$
- Inferenz: Forward-Algorithmus
  $\alpha_k(j) = p(z_k|x_k{=}j)\sum_i \alpha_{k-1}(i)\, A_{ij}$

Kontinuierliches HMM (= Kalman!):

- Transition: $x_k = A\, x_{k-1} + w_k$
  $\Leftrightarrow\ p(x_k|x_{k-1}) = \mathcal{N}(Ax_{k-1},\ Q)$
- Emission: $z_k = H\, x_k + v_k$
  $\Leftrightarrow\ p(z_k|x_k) = \mathcal{N}(Hx_k,\ R)$
- Inferenz: Kalman-Filter
  Summe $\rightarrow$ Integral, exakt lösbar!

Der entscheidende Zusammenhang

	Diskret	Kontinuierlich (Gauß)
Zustand	endlich: $x \in \{1, \ldots, N\}$	$x \in \mathbb{R}^n$
Forward-Variable	$\alpha_k(j)$ (Vektor)	$\mathcal{N}(\hat{x}_k, P_k)$
Prädiktion	Matrix-Vektor $A\,\alpha$	$\hat{x}^- = A\hat{x},\ P^- = APA^T + Q$
Update	$\alpha \cdot B$ (elementweise)	Kalman-Gain K, Innovation

Bayes-Filter – Das allgemeine Prinzip

Grundidee: Zustandsschätzung als probabilistisches Schließen.

Gegeben: bisherige Schätzung $p(\mathrm{x}_{k-1}|\mathrm{z}_{1:k-1})$, neue Messung z_k.

Bayes-Filter in zwei Schritten:

1. Prädiktion (Chapman-Kolmogorov):

$$p(\mathrm{x}_k|\mathrm{z}_{1:k-1}) = \int p(\mathrm{x}_k|\mathrm{x}_{k-1})\; p(\mathrm{x}_{k-1}|\mathrm{z}_{1:k-1})\; d\mathrm{x}_{k-1}$$

2. Update (Bayes'sche Regel):

$$p(\mathrm{x}_k|\mathrm{z}_{1:k}) = \frac{p(\mathrm{z}_k|\mathrm{x}_k)\; p(\mathrm{x}_k|\mathrm{z}_{1:k-1})}{p(\mathrm{z}_k|\mathrm{z}_{1:k-1})}$$

Bayes-Filter – Das allgemeine Prinzip

Grundidee: Zustandsschätzung als probabilistisches Schließen.

Gegeben: bisherige Schätzung $p(\mathrm{x}_{k-1}|\mathrm{z}_{1:k-1})$, neue Messung z_k.

Bayes-Filter in zwei Schritten:

1. Prädiktion (Chapman-Kolmogorov):

$$p(\mathrm{x}_k|\mathrm{z}_{1:k-1}) = \int p(\mathrm{x}_k|\mathrm{x}_{k-1})\; p(\mathrm{x}_{k-1}|\mathrm{z}_{1:k-1})\; d\mathrm{x}_{k-1}$$

2. Update (Bayes'sche Regel):

$$p(\mathrm{x}_k|\mathrm{z}_{1:k}) = \frac{p(\mathrm{z}_k|\mathrm{x}_k)\; p(\mathrm{x}_k|\mathrm{z}_{1:k-1})}{p(\mathrm{z}_k|\mathrm{z}_{1:k-1})}$$

Kalman-Filter = Bayes-Filter mit zwei Annahmen

1. Lineares Systemmodell: $\mathrm{x}_k = A\mathrm{x}_{k-1} + B\mathrm{u}_k + \mathrm{w}_k$
2. Gauß'sches Rauschen: $\mathrm{w}_k \sim \mathcal{N}(0, Q), \quad \mathrm{v}_k \sim \mathcal{N}(0, R)$

Unter diesen Annahmen bleibt die Verteilung *immer* Gauß'sch – die Integrale lassen sich *analytisch* lösen!

Kalman-Filter – Das Systemmodell

Ausgangspunkt: Lineares zeitdiskretes System mit Rauschen.

Zustandsgleichung (Prozessmodell)

$$\mathrm{x}_k = A\,\mathrm{x}_{k-1} + B\,\mathrm{u}_k + \mathrm{w}_k \quad \text{mit } \mathrm{w}_k \sim \mathcal{N}(0, Q)$$

Messgleichung (Beobachtungsmodell)

$$\mathrm{z}_k = H\,\mathrm{x}_k + \mathrm{v}_k \quad \text{mit } \mathrm{v}_k \sim \mathcal{N}(0, R)$$

Die Matrizen im Detail:

$\mathrm{x}_k \in \mathbb{R}^n$	Zustandsvektor	(was wir schätzen wollen)
$\mathrm{u}_k \in \mathbb{R}^m$	Steuervektor	(bekannte Eingabe)
$\mathrm{z}_k \in \mathbb{R}^p$	Messvektor	(was der Sensor liefert)
$A \in \mathbb{R}^{n\times n}$	Systemmatrix	(wie sich der Zustand entwickelt)
$B \in \mathbb{R}^{n\times m}$	Steuermatrix	(Einfluss der Steuerung)
$H \in \mathbb{R}^{p\times n}$	Messmatrix	(Zusammenhang Zustand $\leftrightarrow$ Messung)

Was ist eine Kovarianzmatrix?
Für einen Zufallsvektor $\mathrm{x} = (x_1, \ldots, x_n)^T$:

$$P = \mathrm{Cov}[\mathrm{x}] = \mathrm{E}[(\mathrm{x} - \boldsymbol{\mu})(\mathrm{x} - \boldsymbol{\mu})^T] = \begin{bmatrix} \sigma^2_{x_1} & \mathrm{cov}(x_1, x_2) & \cdots \\ \mathrm{cov}(x_2, x_1) & \sigma^2_{x_2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Drei Kovarianzmatrizen im Kalman-Filter:

- $P_k \in \mathbb{R}^{n \times n}$: Schätzunsicherheit – wie unsicher ist unser aktueller Zustandsschätzer? Diagonale = Varianz jeder Zustandsgröße.
- $Q \in \mathbb{R}^{n \times n}$: Prozessrauschen – wie ungenau ist unser Modell? Größeres $Q \Rightarrow$ Filter vertraut dem Modell weniger.
- $R \in \mathbb{R}^{p \times p}$: Messrauschen – wie ungenau ist der Sensor? Größeres $R \Rightarrow$ Filter vertraut der Messung weniger.

Kalman-Filter – Herleitung der Prädiktion

Schritt 1: Vorhersage des Zustands.
Wenn wir das Systemmodell kennen, können wir vorhersagen:

$$\hat{\mathrm{x}}_{k|k-1} = A\,\hat{\mathrm{x}}_{k-1} + B\,\mathrm{u}_k$$

Wie entwickelt sich die Unsicherheit?

Kalman-Filter – Herleitung der Prädiktion

Schritt 1: Vorhersage des Zustands.
Wenn wir das Systemmodell kennen, können wir vorhersagen:

$$\hat{\mathrm{x}}_{k|k-1} = A\,\hat{\mathrm{x}}_{k-1} + B\,\mathrm{u}_k$$

Wie entwickelt sich die Unsicherheit?
Die Kovarianz transformiert sich durch lineare Abbildung A als:

$$\mathrm{Cov}[A\mathrm{x}] = A\,\mathrm{Cov}[\mathrm{x}]\,A^T$$

(Dies folgt direkt aus $\mathrm{E}[(A\mathrm{x})(A\mathrm{x})^T] = A\,\mathrm{E}[\mathrm{x}\mathrm{x}^T]\,A^T$.)

Schritt 1: Vorhersage des Zustands.
Wenn wir das Systemmodell kennen, können wir vorhersagen:

$$\hat{\mathrm{x}}_{k|k-1} = A\,\hat{\mathrm{x}}_{k-1} + B\,\mathrm{u}_k$$

Wie entwickelt sich die Unsicherheit?
Die Kovarianz transformiert sich durch lineare Abbildung A als:

$$\mathrm{Cov}[A\mathrm{x}] = A\,\mathrm{Cov}[\mathrm{x}]\,A^T$$

(Dies folgt direkt aus $\mathrm{E}[(A\mathrm{x})(A\mathrm{x})^T] = A\,\mathrm{E}[\mathrm{x}\mathrm{x}^T]\,A^T$.)
Dazu kommt das Prozessrauschen Q:

$$\boxed{P_{k|k-1} = A\,P_{k-1}\,A^T + Q}$$

Intuition

Die Unsicherheit *wächst* bei jeder Prädiktion – wir verlieren Information, weil das Modell nicht perfekt ist.

Kalman-Filter – Herleitung des Updates

Schritt 2: Korrektur mit der Messung.
Die Innovation (Überraschung der Messung):

$$\tilde{y}_k = z_k - H\hat{x}_{k|k-1}$$

Kalman-Filter – Herleitung des Updates

Schritt 2: Korrektur mit der Messung.
Die Innovation (Überraschung der Messung):

$$\tilde{y}_k = z_k - H\,\hat{x}_{k|k-1}$$

Innovationskovarianz (wie unsicher ist die Überraschung?):

$$S_k = H\,P_{k|k-1}\,H^T + R$$

Kalman-Filter – Herleitung des Updates

Schritt 2: Korrektur mit der Messung.
Die Innovation (Überraschung der Messung):

$$\tilde{y}_k = z_k - H\,\hat{x}_{k|k-1}$$

Innovationskovarianz (wie unsicher ist die Überraschung?):

$$S_k = H\,P_{k|k-1}\,H^T + R$$

Kalman-Gain (optimale Gewichtung):

$$\boxed{K_k = P_{k|k-1}\,H^T\,S_k^{-1}}$$

Kalman-Filter – Herleitung des Updates

Schritt 2: Korrektur mit der Messung.
Die Innovation (Überraschung der Messung):

$$\tilde{y}_k = z_k - H\,\hat{x}_{k|k-1}$$

Innovationskovarianz (wie unsicher ist die Überraschung?):

$$S_k = H\,P_{k|k-1}\,H^T + R$$

Kalman-Gain (optimale Gewichtung):

$$\boxed{K_k = P_{k|k-1}\,H^T\,S_k^{-1}}$$

Update-Gleichungen:

$$\boxed{\hat{x}_k = \hat{x}_{k|k-1} + K_k\,\tilde{y}_k} \qquad \boxed{P_k = (I - K_k\,H)\,P_{k|k-1}}$$

Der Kalman-Gain minimiert den mittleren quadratischen Fehler $\mathrm{E}[\|x_k - \hat{x}_k\|^2]$ – er ist optimal für lineare Systeme mit Gauß'schem Rauschen.

Kalman-Filter – Intuition des Kalman-Gain

Was macht $K_k = P_{k|k-1}\, H^T\, (H\, P_{k|k-1}\, H^T + R)^{-1}$?

Fall 1: R klein (Sensor genau)

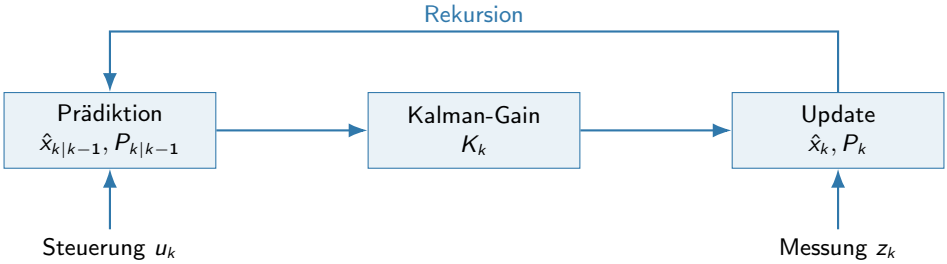
- $S_k \approx HP_{k|k-1}H^T$
- K_k wird groß
- $\hat{x}_k \approx$ Messung

$\Rightarrow$ Filter vertraut dem Sensor.

Fall 2: $P_{k|k-1}$ klein (Modell sicher)

- Zähler $P_{k|k-1}H^T$ wird klein
- $K_k \to 0$
- $\hat{x}_k \approx$ Prädiktion

$\Rightarrow$ Filter vertraut dem Modell.



Kalman-Filter – Turtlebot Systemmodell

Zustand: $\mathrm{x} = (x, y, \theta, v, \omega)^T$ (Position, Orientierung, Geschwindigkeiten)

Prozessmodell (Differential Drive, Δt klein):

$$\mathrm{x}_{k+1} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & \cos\theta_k\,\Delta t & 0 \\ 0 & 1 & 0 & \sin\theta_k\,\Delta t & 0 \\ 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{f(\mathrm{x}_k)} \mathrm{x}_k + \mathrm{w}_k$$

Zustand: $\mathrm{x} = (x, y, \theta, v, \omega)^T$ (Position, Orientierung, Geschwindigkeiten)

Prozessmodell (Differential Drive, Δt klein):

$$\mathrm{x}_{k+1} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & \cos\theta_k\,\Delta t & 0 \\ 0 & 1 & 0 & \sin\theta_k\,\Delta t & 0 \\ 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{f(\mathrm{x}_k)} \mathrm{x}_k + \mathrm{w}_k$$

Nichtlinearität!

Matrix hängt von θ_k ab $\Rightarrow$ nichtlinear – Standard-Kalman nicht direkt anwendbar!

Messmodell (Encoder: v, ω; Lidar: x, y): $\mathrm{z}_k = H\,\mathrm{x}_k + \mathrm{v}_k$ mit $H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$

Turtlebot – Kovarianzmatrizen konkret

Prozessrauschen Q – Unsicherheit des Bewegungsmodells:

$$Q = \mathrm{diag}\big(\sigma_x^2,\ \sigma_y^2,\ \sigma_\theta^2,\ \sigma_v^2,\ \sigma_\omega^2\big)$$

Typisch: $\sigma_x = \sigma_y \approx 0.01\,\mathrm{m}, \quad \sigma_\theta \approx 0.01\,\mathrm{rad}, \quad \sigma_v \approx 0.05\,\mathrm{m/s}$

Messrauschen R – Unsicherheit der Sensoren:

$$R = \mathrm{diag}\big(\sigma_{x,\mathrm{lidar}}^2,\ \sigma_{y,\mathrm{lidar}}^2,\ \sigma_{v,\mathrm{enc}}^2,\ \sigma_{\omega,\mathrm{enc}}^2\big)$$

Typisch: $\sigma_{\mathrm{lidar}} \approx 0.05\,\mathrm{m}, \quad \sigma_{\mathrm{enc}} \approx 0.02\,\mathrm{m/s}$

Tuning-Faustregel

Q und R werden aus Sensor-Datenblättern und experimentellen Messungen bestimmt. Das Verhältnis Q/R bestimmt, ob der Filter eher dem Modell oder den Sensoren vertraut.

Extended Kalman Filter (EKF)

Problem: Turtlebot-Modell ist nichtlinear: $\mathrm{x}_{k+1} = f(\mathrm{x}_k, \mathrm{u}_k) + \mathrm{w}_k$

Idee: Linearisiere f in jedem Zeitschritt um den aktuellen Schätzwert $\hat{\mathrm{x}}_k$ mittels Jacobi-Matrix:

$$F_k = \left.\frac{\partial f}{\partial \mathrm{x}}\right|_{\hat{\mathrm{x}}_k} = \begin{bmatrix} 1 & 0 & -v_k \sin\theta_k \,\Delta t & \cos\theta_k \,\Delta t & 0 \\ 0 & 1 & v_k \cos\theta_k \,\Delta t & \sin\theta_k \,\Delta t & 0 \\ 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Extended Kalman Filter (EKF)

Problem: Turtlebot-Modell ist nichtlinear: $\mathrm{x}_{k+1} = f(\mathrm{x}_k, \mathrm{u}_k) + \mathrm{w}_k$

Idee: Linearisiere f in jedem Zeitschritt um den aktuellen Schätzwert $\hat{\mathrm{x}}_k$ mittels Jacobi-Matrix:

$$F_k = \left.\frac{\partial f}{\partial \mathrm{x}}\right|_{\hat{\mathrm{x}}_k} = \begin{bmatrix} 1 & 0 & -v_k \sin\theta_k \,\Delta t & \cos\theta_k \,\Delta t & 0 \\ 0 & 1 & v_k \cos\theta_k \,\Delta t & \sin\theta_k \,\Delta t & 0 \\ 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

EKF-Gleichungen (wie Kalman, aber $A \to F_k$):

$$\begin{aligned} \text{Prädiktion:} \quad & \hat{\mathrm{x}}_{k|k-1} = f(\hat{\mathrm{x}}_{k-1}, \mathrm{u}_k), \quad P_{k|k-1} = F_k \, P_{k-1} \, F_k^T + Q \\ \text{Update:} \quad & K_k = P_{k|k-1} \, H^T \left(H \, P_{k|k-1} \, H^T + R\right)^{-1} \\ & \hat{\mathrm{x}}_k = \hat{\mathrm{x}}_{k|k-1} + K_k \left(z_k - H \, \hat{\mathrm{x}}_{k|k-1}\right) \end{aligned}$$

EKF – Zusammenfassung und Grenzen

Vorteile des EKF:

- Funktioniert für nichtlineare Systeme
- Standardverfahren in der Robotik
- Effizient (nur Matrixoperationen)
- Gut verstanden, viele Bibliotheken

Grenzen des EKF:

- Linearisierung nur lokal gültig
- Kann bei starker Nichtlinearität divergieren
- Gauß-Annahme kann verletzt sein
- Jacobi-Matrizen analytisch nötig

Alternativen bei starker Nichtlinearität

- UKF: Sigma-Punkte statt Linearisierung
- Partikelfilter: Monte-Carlo-Sampling, keine Gauß-Annahme nötig

Complementary Filter (IMU)

Problem: Gyroskop liefert gute *kurzfristige* Orientierung (aber driftet), Beschleunigungssensor liefert stabile *langfristige* Neigung (aber verrauscht).

Complementary Filter

$$\theta_k = \alpha \cdot (\theta_{k-1} + \omega_k \cdot \Delta t) + (1 - \alpha) \cdot \theta_{\text{acc,k}}$$

mit $\alpha \approx 0.98$ (High-Pass auf Gyro, Low-Pass auf Beschleunigung).

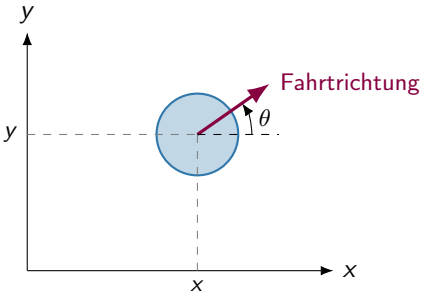
- Vorteil: Extrem einfach, kaum Rechenaufwand
- Nachteil: Nicht optimal, kein Modell der Unsicherheit
- Einsatz: Drohnen, Turtlebot, schnelle Prototypen

Zustandsschätzung

Pose eines Roboters

Die Pose beschreibt Position und Orientierung in der Ebene:

$$\mathsf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \in \mathbb{R}^2 \times (-\pi, \pi]$$



Warum reicht (x, y) nicht?

- Roboter muss wissen, *wohin* er schaut
- Steuerung hängt von θ ab
- Sensoren sind richtungsabhängig (Kamera, Lidar)

Odometrie – Wheel Encoder

Idee: Aus der Raddrehung auf die zurückgelegte Strecke schließen.

Encoder-Modell:

- Ticks pro Umdrehung: N_{ticks}
- Radradius: r
- Strecke pro Tick: $\Delta s = \frac{2\pi r}{N_{\text{ticks}}}$

Differentielle Odometrie:

$$\Delta s = \frac{\Delta s_L + \Delta s_R}{2}$$

$$\Delta\theta = \frac{\Delta s_R - \Delta s_L}{L}$$

(L = Achsabstand)

Problem: Drift!

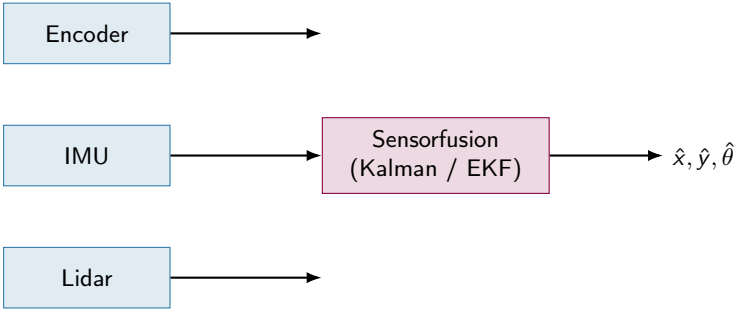
Odometrie akkumuliert Fehler über die Zeit:

- Radschlupf
- Ungleiche Raddurchmesser
- Bodenunebenheiten

$\Rightarrow$ Sensorfusion nötig!

Sensorfusion

Ziel: Mehrere Sensoren kombinieren, um die Schwächen einzelner Sensoren auszugleichen.



- Encoder: hochfrequent, aber driftet
- IMU: schnelle Dynamik, aber Gyro-Drift
- Lidar: absolut genau, aber niedrige Rate

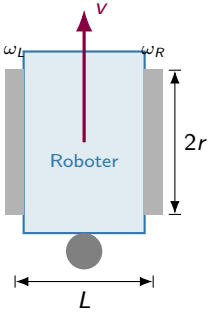
$\Rightarrow$ Kalman-Filter gewichtet je nach Unsicherheit.

Kinematik & Bewegungsmodelle

Differential Drive (Turtlebot)

Aufbau:

- Zwei unabhängig angetriebene Räder
- Stützrad (passiv)
- Parameter: Radradius r, Achsabstand L



Geschwindigkeiten:

$$v = \frac{r}{2}(\omega_R + \omega_L)$$
$$\omega = \frac{r}{L}(\omega_R - \omega_L)$$

ω_R, ω_L: Winkelgeschwindigkeiten der Räder

Forward & Inverse Kinematics

Forward Kinematics:
$v, \omega \;\rightarrow\; x, y, \theta$

Diskretisiert (Δt klein):

$$
\begin{aligned}
x_{k+1} &= x_k + v_k \cos(\theta_k)\,\Delta t \\
y_{k+1} &= y_k + v_k \sin(\theta_k)\,\Delta t \\
\theta_{k+1} &= \theta_k + \omega_k\,\Delta t
\end{aligned}
$$

Gegeben: Rad-Geschwindigkeiten
Gesucht: resultierende Pose

Inverse Kinematics:
Gewünschte Bewegung $\rightarrow$
Radgeschwindigkeiten

$$
\omega_R = \frac{v + \frac{L}{2}\,\omega}{r}
$$

$$
\omega_L = \frac{v - \frac{L}{2}\,\omega}{r}
$$

Gegeben: gewünschtes (v, ω)
Gesucht: Motoransteuerung (ω_L, ω_R)

Regelung (Control)

Ohne Zustandsschätzung kein stabiler Regler

Warum hängen Estimation und Control zusammen?

Der PID-Regler braucht den aktuellen Zustand als Eingabe:

$$e(t) = x_{\text{soll}} - \underbrace{x_{\text{ist}}}_{\text{woher?}}$$

Aber x_{ist} ist *nie* direkt messbar – nur über verrauschte Sensoren!

Ohne Zustandsschätzung kein stabiler Regler

Warum hängen Estimation und Control zusammen?

Der PID-Regler braucht den aktuellen Zustand als Eingabe:

$$e(t) = x_{\text{soll}} - \underbrace{x_{\text{ist}}}_{\text{woher?}}$$

Aber x_{ist} ist *nie* direkt messbar – nur über verrauschte Sensoren!

Ohne Filter (rohe Sensordaten):

- Rauschen geht direkt in $e(t)$
- $K_d \cdot \frac{de}{dt}$ verstärkt Rauschen!
- Regler oszilliert oder wird instabil

Mit Kalman-Filter:

- Geglättete, optimale Schätzung $\hat{x}$
- Auch Geschwindigkeit $\hat{v}$ geschätzt (nicht numerisch differenziert)
- Regler bekommt saubere Eingabe

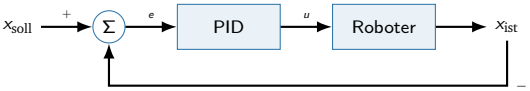
Separation Principle (LQG)

Für lineare Systeme mit Gauß'schem Rauschen kann man Zustandsschätzung (Kalman) und Regelung (LQR/PID) getrennt entwerfen – das Gesamtsystem bleibt optimal.

PID-Regelgesetz

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d\, \frac{de(t)}{dt}$$

mit Regelabweichung $e(t) = x_{\text{soll}}(t) - x_{\text{ist}}(t)$



- K_p: Proportional – reagiert auf aktuellen Fehler
- K_i: Integral – beseitigt stationären Fehler
- K_d: Differential – dämpft Überschwinger

Trajektorienverfolgung

Aufgabe: Roboter soll einem Pfad folgen oder einen Zielpunkt (x_g, y_g) erreichen.

Einfache Strategie ("go-to-goal"):

1. Berechne Richtung zum Ziel: $\alpha = \text{atan2}(y_g - y,\ x_g - x)$
2. Regelfehler Orientierung: $e_\theta = \alpha - \theta$
3. PID auf e_θ liefert ω
4. v konstant oder abhängig von Distanz

Stabilität

- Overshoot: Roboter schießt über Ziel hinaus (K_p zu groß)
- Oszillation: Pendeln um Zielpunkt (K_d zu klein)
- Stationärer Fehler: Ziel wird nie ganz erreicht ($K_i = 0$)

Lokalisierung & Mapping (SLAM)

Das Lokalisierungsproblem

Zwei fundamentale Fragen:

1. Wo bin ich? (Lokalisierung)
2. Wie sieht die Welt aus? (Mapping)

Henne-Ei-Problem:

- Lokalisierung braucht Karte
- Kartenbau braucht Position
- $\Rightarrow$ beides gleichzeitig: SLAM

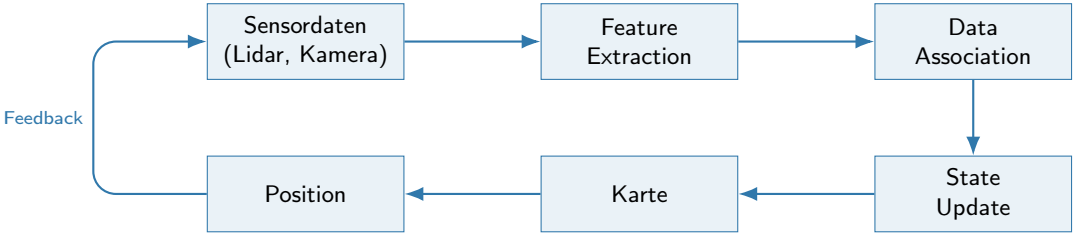
Landmarken:

- Markante Punkte in der Umgebung
- Wiedererkennbar (Ecken, Kanten, Objekte)
- Distanz + Winkel $\Rightarrow$ Triangulation

Occupancy Grid Map:

- Raster: jede Zelle frei / belegt / unbekannt
- Aufgebaut aus Lidar-Scans
- Grundlage für Pfadplanung

SLAM – Simultaneous Localization and Mapping



- Feature Extraction: Landmarken aus Sensordaten extrahieren
- Data Association: Welche beobachtete Landmarke entspricht welcher bekannten?
- State Update: Karte und Position gemeinsam korrigieren (EKF-SLAM, Partikelfilter, Graph-SLAM)

Vom EKF zu SLAM – der wachsende Zustandsvektor

EKF-Lokalisierung (bekannte Karte): $\mathrm{x} = (x, y, \theta)^T \in \mathbb{R}^3$ – feste Dimension.

Vom EKF zu SLAM – der wachsende Zustandsvektor

EKF-Lokalisierung (bekannte Karte): $\mathrm{x} = (x, y, \theta)^T \in \mathbb{R}^3$ – feste Dimension.

EKF-SLAM (unbekannte Karte): Landmarken werden Teil des Zustands!

$$\mathrm{x}_{\mathrm{SLAM}} = \begin{bmatrix} x_R \\ y_R \\ \theta_R \\ \hline x_{L_1} \\ y_{L_1} \\ \hline x_{L_2} \\ y_{L_2} \\ \hline \vdots \\ x_{L_M} \\ y_{L_M} \end{bmatrix} \in \mathbb{R}^{3+2M}$$

Jede neue Landmarke erweitert den Vektor um 2 Einträge!

Vom EKF zu SLAM – der wachsende Zustandsvektor

EKF-Lokalisierung (bekannte Karte): $\mathrm{x} = (x, y, \theta)^T \in \mathbb{R}^3$ – feste Dimension.

EKF-SLAM (unbekannte Karte): Landmarken werden Teil des Zustands!

$$\mathrm{x}_{\mathrm{SLAM}} = \begin{bmatrix} x_R \\ y_R \\ \theta_R \\ \hline x_{L_1} \\ y_{L_1} \\ \hline x_{L_2} \\ y_{L_2} \\ \hline \vdots \\ x_{L_M} \\ y_{L_M} \end{bmatrix} \in \mathbb{R}^{3+2M}$$

Jede neue Landmarke erweitert den Vektor um 2 Einträge!

Skalierungsproblem

$P \in \mathbb{R}^{(3+2M)\times(3+2M)}$ wächst quadratisch! Bei M=1000: $>$4 Mio. Einträge $\Rightarrow$ EKF-SLAM für große Umgebungen zu langsam. Alternativen: Partikelfilter-SLAM, Graph-SLAM.

SLAM-Zustand: Kovarianzstruktur

Die vollständige Kovarianzmatrix:

$$P = \begin{bmatrix} P_{RR} & P_{RL_1} & P_{RL_2} & \cdots & P_{RL_M} \\ P_{L_1R} & P_{L_1L_1} & P_{L_1L_2} & \cdots & P_{L_1L_M} \\ P_{L_2R} & P_{L_2L_1} & P_{L_2L_2} & \cdots & \vdots \\ \vdots & & & \ddots & \\ P_{L_MR} & \cdots & & & P_{L_ML_M} \end{bmatrix}$$

- P_{RR}: Unsicherheit der Roboterpose (wie beim EKF)
- $P_{L_iL_i}$: Unsicherheit der i-ten Landmarke
- P_{RL_i}: Kreuzkorrelation zwischen Roboter und Landmarke – *das* ist das Entscheidende! Beobachtung einer Landmarke verbessert die Schätzung *aller* korrelierter Landmarken.

Loop Closure

Wenn der Roboter einen bereits besuchten Ort wiedererkennt, können über die Kreuzkorrelationen Fehler in der gesamten Karte korrigiert werden.

Pfad- und Trajektorienplanung

Pfadplanung – A* und Dijkstra

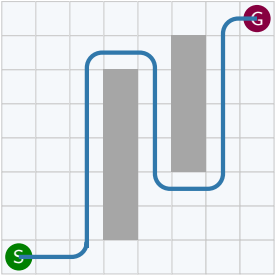
Gegeben: Occupancy Grid Map, Start s, Ziel g.
Gesucht: Kürzester hindernisfreier Pfad.

Dijkstra:

- Expandiert alle Knoten nach Kosten
- Garantiert optimalen Pfad
- Keine Richtungsinformation
  $\Rightarrow$ langsam bei großen Karten

A*:

- Erweitert Dijkstra um Heuristik $h(n)$
- $f(n) = g(n) + h(n)$
- Typisch: $h =$ euklidische Distanz zum Ziel
- Optimal, wenn h zulässig (nicht überschätzt)



Trajektorienplanung & Hindernisvermeidung

Pfad $\neq$ Trajektorie:

- Pfad: geometrische Kurve $(x(s), y(s))$
- Trajektorie: Pfad + Zeitparametrisierung $(x(t), y(t), \theta(t))$

Anforderungen an die Trajektorie:

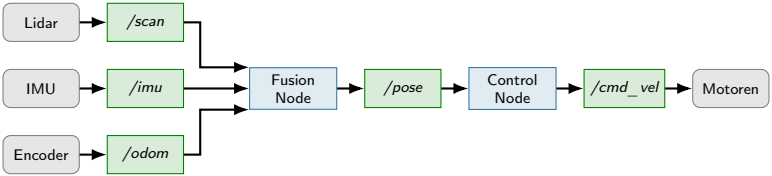
1. Glatt: stetig differenzierbar (keine Sprünge in v oder ω)
2. Physikalisch realisierbar: $v_{\max}$, $\omega_{\max}$ einhalten
3. Hindernisfrei: Sicherheitsabstand zu Objekten

Hindernisvermeidung (reaktiv):

- Bug-Algorithmen: Hindernis umfahren, dann Zielrichtung wieder aufnehmen
- Potentialfeldmethode: Ziel = anziehend, Hindernisse = abstoßend
- Dynamic Window Approach (DWA): Suche im (v, ω)-Raum

Softwarearchitektur (ROS 2)

Sensor → Topic → Node → Control Loop



Jede Kante ist ein ROS 2 Topic – lose Kopplung, austauschbare Komponenten.

Publisher / Subscriber:

- Asynchrone Kommunikation
- Publisher sendet auf Topic
- Subscriber empfängt Callback
- Kein direkter Aufruf ⇒ Entkopplung

Quality of Service (QoS):

- `RELIABLE`: jede Nachricht kommt an (TCP-ähnlich)
- `BEST_EFFORT`: keine Garantie (UDP-ähnlich, weniger Latenz)
- `KEEP_LAST(n)`: nur die *n* neuesten Nachrichten puffern

Wichtig für Echtzeit!

Falsche QoS-Einstellungen können zu:

- verlorenen Sensordaten
- erhöhter Latenz
- nicht-deterministischem Verhalten

führen. ⇒ QoS bewusst wählen!

Typische Kombination:

- Sensordaten: `BEST_EFFORT`
- Steuerbefehle: `RELIABLE`

Timing: Frequenzen und Latenz

Verschiedene Raten im System:

Komponente	Rate	Typ
IMU	200 Hz	Sensor
Lidar	10 Hz	Sensor
Kamera	30 Hz	Sensor
Odometrie	50 Hz	Verarbeitung
Regelung	20 Hz	Control
SLAM	1 Hz	Planung

Latenz-Budget:

- Gesamtlatenz: Sensor $\rightarrow$ Aktor
- Muss kleiner als Regelungsperiode sein
- Bei 20 Hz Control: Budget < 50 ms

ROS 2 Timer

```
timer = create_wall_timer(
  50ms, callback);
```

Echtzeit und Embedded-Aspekte

Sampling Rates vs. Control Rates

Kernunterscheidung:

- Sampling Rate: Wie oft wird gemessen? (Sensorfrequenz)
- Control Rate: Wie oft wird geregelt? (Aktorfrequenz)

In der Regel gilt: $f_{\text{sensor}} \geq f_{\text{control}} \geq f_{\text{planning}}$

Beispiel Turtlebot

IMU Sampling:	100 Hz
Encoder Sampling:	50 Hz
PID Control Loop:	20 Hz
Pfadplanung:	1 Hz

Jede Schicht konsumiert die Daten der darüberliegenden.

Determinismus, Interrupts, Polling

Determinismus:

- Regelschleife muss garantiert innerhalb der Deadline fertig sein
- Linux ist *nicht* echtzeitfähig (aber: PREEMPT_RT Patch)
- Microcontroller: oft bare-metal oder RTOS

Interrupts vs. Polling:

- Interrupt: Hardware signalisiert Ereignis ⇒ sofortige Reaktion
- Polling: Software fragt zyklisch ab ⇒ einfacher, aber Latenz

Hardware-nahe Verarbeitung (C):

- Direkte Registerzugriffe
- Kein dynamisches Speichermanagement
- Vorhersagbare Ausführungszeit

```c
// Interrupt Service Routine
void IMU_IRQHandler(void) {
    int16_t raw = IMU->DATA_REG;
    ring_buffer_push(&imu_buf, raw);
    // Keine malloc, printf etc.!
}

// Polling im Main-Loop
while (1) {
    if (timer_elapsed(20ms)) {
        run_pid_controller();
    }
}

```

Unsicherheit & Verifikation

Umgang mit Unsicherheit

In der Robotik ist nichts exakt:

- Sensoren rauschen (Gaussian Noise, Bias)
- Modelle sind Approximationen (Radschlupf, Linearisierung)
- Die Umgebung ist dynamisch und teilweise unbekannt

Konsequenz für den Entwurf:

1. Probabilistische Modelle: Zustand als Wahrscheinlichkeitsverteilung, nicht als Punkt
2. Robuste Regelung: Sicherheitsmargen in Geschwindigkeit und Abstand
3. Defensive Planung: Worst-Case-Szenarien einplanen

Safety-Anforderung (Beispiel)

"Der Roboter hält *immer* mindestens 30 cm Abstand zu jedem Hindernis" – unabhängig von Sensorrauschen.

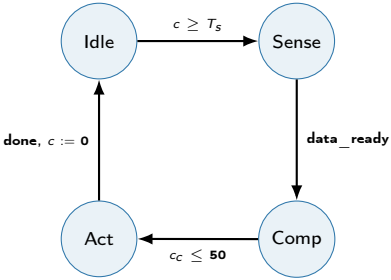
Rückgriff auf die Modellierungstechniken der Vorlesung!

Timed Automata für Control Loops:

- Zustände: `Idle`, `Sensing`, `Computing`, `Actuating`
- Uhren: c_{sense}, c_{ctrl}
- Guards: $c_{\text{ctrl}} \leq 50\,\text{ms}$
- Invarianten erzwingen Deadlines

Safety-Eigenschaften:

- "Control-Schleife terminiert immer innerhalb von 50 ms"
- "Sensordaten sind nie älter als 100 ms"



$c_C \leq 50$: Invariante

Verifikation mit UPPAAL

UPPAAL (bekannt aus der Vorlesung) eignet sich hervorragend für die Verifikation von Robotik-Scheduling:

Modellierung:

- Jede Task (Sensor, Filter, Control) als Timed Automaton
- Kommunikation über Channels
- Shared Variables für Sensordaten

Verifikation (CTL):

- `A[] not deadlock`
  (kein Deadlock möglich)
- `A[] ctrl.active imply c_ctrl <= 50`
  (Deadline wird immer eingehalten)
- `A[] distance >= 30`
  (Sicherheitsabstand garantiert)

Bezug zur Vorlesung

- FSMs → Verhalten
- Timed Automata → Timing
- UPPAAL → Verifikation
- C / Rust → Implementierung
- ROS 2 → Integration

Alle Werkzeuge kommen hier zusammen!

Wie verifiziert man probabilistische Systeme?

Problem: Kalman, SLAM, RL – alle arbeiten mit Wahrscheinlichkeiten. Klassische Model Checker (UPPAAL) prüfen nur ja/nein-Eigenschaften.

Probabilistic Model Checking:

- Modelle: DTMC, MDP, CTMC (Markov-Ketten mit/ohne Nichtdeterminismus)
- Spezifikation: PCTL / CSL
  z. B. $\mathcal{P}_{\geq 0.99}[\Diamond\, goal]$
  „Ziel wird mit $\geq$ 99% erreicht"
- $\mathcal{P}_{\leq 10^{-6}}[\Diamond\, collision]$
  „Kollision mit $\leq$ 0.0001%"

Werkzeuge:

- PRISM – probabilistischer Model Checker
- Storm – effizient für große Modelle
- UPPAAL SMC – Statistical Model Checking (Monte-Carlo statt exakt)

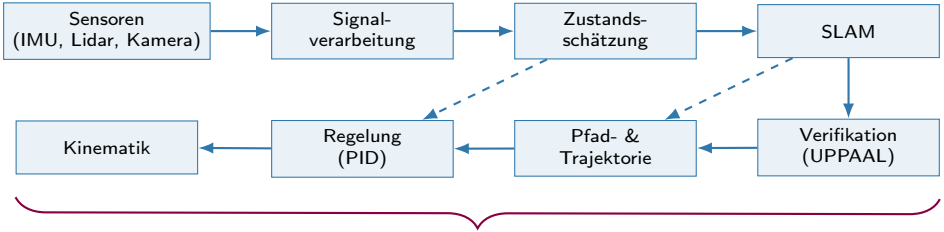
Brücke zu UPPAAL:
UPPAAL SMC kann *stochastische* Timed Automata simulieren und probabilistische Queries beantworten.

Robotik-Anwendung

Sensorrauschen als stochastisches Modell → „Wie wahrscheinlich ist Kollision bei Sensorausfall?" formal prüfbar.

Zusammenfassung

Zusammenfassung: Von Sensoren zur sicheren Robotik



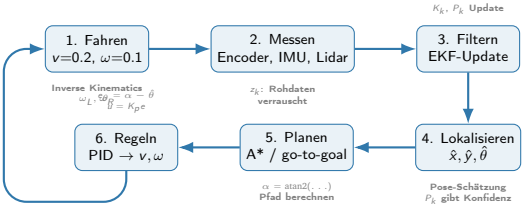
Alles verbunden über ROS 2 – verifizierbar mit UPPAAL

Kernbotschaften

1. Sensoren sind ungenau – Rauschen, Bias und Drift müssen modelliert und gefiltert werden.
2. Kalman-Filter ist das zentrale Werkzeug für Zustandsschätzung unter Unsicherheit.
3. Kinematik verbindet Steuergrößen mit Bewegung – Forward und Inverse Kinematics.
4. PID-Regelung ist der Arbeitspferd-Regler; Stabilität muss beachtet werden.
5. SLAM löst das Henne-Ei-Problem der gleichzeitigen Lokalisierung und Kartierung.
6. Echtzeit ist kein Nice-to-have: Deadlines müssen eingehalten werden.
7. Formale Verifikation (Timed Automata, UPPAAL) schließt den Kreis zur sicheren Implementierung.

Durchgehendes Beispiel: Ein Turtlebot-Zyklus

Szenario: Turtlebot soll von $(0, 0)$ zum Ziel $(3, 2)$ fahren.



Jede Runde dauert $\Delta t = 50\,\text{ms}$ ($20\,\text{Hz}$). Der Zyklus wiederholt sich, bis $\|\hat{\mathrm{p}} - \mathrm{p}_{\text{goal}}\| < \epsilon$.

Turtlebot-Zyklus – Zahlenbeispiel

Zeitschritt $k = 42$: Roboter bei $\hat{x} \approx 1.8$, $\hat{y} \approx 1.1$, $\hat{\theta} \approx 0.5\,\text{rad}$

1. Messen: Encoder: $v_{\text{enc}} = 0.19\,\text{m/s}$ IMU: $\omega_{\text{imu}} = 0.12\,\text{rad/s}$ Lidar: $(x_L, y_L) = (1.83, 1.08)$
2. EKF-Prädiktion ($\Delta t = 0.05\,\text{s}$):

$$\begin{aligned}
\hat{x}^- &= 1.8 + 0.2 \cdot \cos(0.5) \cdot 0.05 = 1.8088 \\
\hat{y}^- &= 1.1 + 0.2 \cdot \sin(0.5) \cdot 0.05 = 1.1048 \\
\hat{\theta}^- &= 0.5 + 0.1 \cdot 0.05 = 0.505
\end{aligned}$$

3. EKF-Update (Lidar-Korrektur): Innovation
   $(1.83 - 1.8088,\ 1.08 - 1.1048) = (0.021, -0.025)$
   $\Rightarrow$ Kalman-Gain korrigiert Pose leicht Richtung Lidar.
4. Planen: Richtung zum Ziel $\alpha = \text{atan2}(2 - 1.10,\ 3 - 1.81) \approx 0.645\,\text{rad}$
5. Regeln: $e_\theta = 0.645 - 0.505 = 0.14 \;\Rightarrow\; \omega = K_p \cdot 0.14 = 0.28\,\text{rad/s}$

$\Rightarrow$ Nächster Schritt: etwas mehr nach links lenken, weiterfahren.

Exkurs: Reinforcement Learning (RL)

Bisher: Modellbasierte Ansätze – wir kennen das physikalische Modell und entwerfen Regler analytisch (PID, Kalman).

Alternative: Was wenn das Modell zu komplex oder unbekannt ist?

Reinforcement Learning

Ein Agent lernt durch Trial-and-Error, welche Aktion a in welchem Zustand s die höchste kumulative Belohnung R liefert.

Formal: Markov Decision Process (MDP) (S, A, P, R, γ).

RL – Grundbegriffe

Ziel: Finde eine Policy $\pi(a|s)$, die den erwarteten diskontierten Gesamtnutzen maximiert:

$$J(\pi) = \mathrm{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \, r_{t+1} \right]$$

mit Diskontfaktor $\gamma \in [0, 1)$.

Kernkonzepte:

- Value-Funktion $V^\pi(s)$: erwarteter Gesamtnutzen ab Zustand s

$$V^\pi(s) = \mathrm{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \;\middle|\; s_0 = s \right]$$

- Q-Funktion $Q^\pi(s, a)$: wie V, aber zusätzlich bedingt auf Aktion a
- Bellman-Gleichung: $Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) \, V^\pi(s')$
- Exploration vs. Exploitation: Neues ausprobieren vs. bekannte gute Strategien nutzen

RL-Algorithmen – Überblick

Tabellarische Methoden:

- Q-Learning:
  Lerne $Q(s, a)$-Tabelle iterativ
- Update: $Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$
- Konvergiert zum Optimum
- Nur für kleine, diskrete Zustands-/Aktionsräume

Für Robotik:

- Zustandsraum ist *kontinuierlich* $(x, y, \theta, v, \omega, \ldots)$
- Aktionsraum ist *kontinuierlich* $(v_{\text{cmd}}, \omega_{\text{cmd}})$
- Q-Tabelle unmöglich (unendlich viele Einträge)
- $\Rightarrow$ Funktionsapproximation nötig

Motivation für Deep RL

Neuronale Netze als universelle Funktionsapproximatoren können $Q(s, a)$ oder $\pi(a|s)$ für kontinuierliche Räume lernen.

Deep Reinforcement Learning

Idee: Ersetze die Q-Tabelle / Policy durch ein neuronales Netz mit Parametern θ.

Value-based (DQN):

- Netz approximiert $Q_\theta(s, a)$
- Trainiert mit Experience Replay
- Funktioniert für diskrete Aktionen
- Durchbruch: Atari-Spiele (2015)

Policy-based:

- Netz gibt direkt $\pi_\theta(a|s)$ aus
- Gradient: $\nabla_\theta J \approx \mathrm{E}[\nabla_\theta \log \pi_\theta \cdot R]$
- Funktioniert für kontinuierliche Aktionen

Actor-Critic (z. B. PPO, SAC):

- Actor: Policy-Netz $\pi_\theta(a|s)$
- Critic: Value-Netz $V_\phi(s)$
- Critic bewertet die Aktionen des Actors
- Stabiler als reines Policy Gradient

Für Robotik besonders relevant:

- PPO (Proximal Policy Optimization)
- SAC (Soft Actor-Critic)
- TD3 (Twin Delayed DDPG)

Deep RL in der Robotik – Chancen und Grenzen

Wo Deep RL gut funktioniert:

- Laufroboter (Locomotion)
- Greifplanung (Manipulation)
- Hindernisvermeidung in unbekannter Umgebung
- Wenn physikalisches Modell zu komplex oder unbekannt

Sim-to-Real:
Training in Simulation, Transfer auf realen Roboter. Spart Millionen von Episoden auf realer Hardware.

Offene Herausforderungen:

- Sample Inefficiency: Millionen Episoden für Training
- Safety: Wie verhindert man gefährliches Verhalten während des Lernens?
- Interpretierbarkeit: Neuronales Netz ist Black Box
- Sim-to-Real Gap: Simulation $\neq$ Realität

Abgrenzung zu modellbasierten Methoden

PID + Kalman + SLAM: interpretierbar, verifizierbar, deterministisch. Deep RL: flexibel, lernfähig, aber schwer zu verifizieren. Praxis: modellbasiert + RL für Feintuning.